FINAL PROJECT DOCUMENTATION

BANKING APPLICATION

BanCU

Java • Spring Boot • REST API • MySQL

PREPARED BY

Student Name: Alexander Inciong

Batch / Section: PRJ13

Trainer: Michael Ampo

Date: 15 October 2026

Official TESDA and CIICC logos may be placed in the reserved cover area.

Table of Contents

1. Project Overview 3

2. System Features 4–5

3. System Design 6

4. Database Design 7

5. Code Snippets 8–9

6. Test Evidence: Postman 10

7. Test Evidence: Console 11

8. Conclusion & Appendix 12

This document describes the supplied source structure and test evidence. The Postman collection run screenshot reports 8 tests passed and 0 failures; the List All Accounts request returned HTTP 200 but has no assertion configured in that run.

1. Project Overview

BanCU is a Java and Spring Boot banking application for creating accounts, checking balances, depositing and withdrawing funds, transferring funds between accounts, viewing paginated transaction history, and listing all accounts. It exposes REST endpoints and includes an optional interactive command-line interface that calls the same service layer.

Purpose and scope

The project demonstrates a layered application design. REST controllers accept requests and validate DTOs; service classes implement banking rules; Spring Data JPA repositories access entities; and MySQL stores account and transaction records. Monetary values use BigDecimal. Money-changing services are transactional and use pessimistic write locks to protect account updates.

Technology profile

Area	Implementation
Language / framework	Java, Spring Boot
Web API	Spring MVC REST controllers
Persistence	Spring Data JPA / Hibernate
Database	MySQL (bancu_db)
Validation / utilities	Jakarta Bean Validation, Lombok
Optional interface	CommandLineRunner console; controlled by bancu.console.enabled

Feature status: List-all-accounts is implemented in AccountController and AccountService, and is available at GET /api/accounts. The supplied Postman run shows this request returned 200.

2. System Features

2.1 Create Account

POST /api/accounts — Creates an ACTIVE account with a unique 12-character uppercase account number. The initial deposit is optional and defaults to zero.

```
Request: { "accountHolder": "Alex", "initialDeposit": 250.00 }
```

2.2 Balance Inquiry

GET /api/accounts/{accountNumber}/balance — Returns account number and current balance. An unknown account is reported as not found.

```
Response: { "accountNumber": "F37358DD9897", "balance": 2000.00 }
```

2.3 List All Accounts

GET /api/accounts — Returns a JSON list of account response objects containing account number, holder, balance, and account status.

```
Response: [ { "accountNumber": "…", "accountHolder": "Alex", "balance": 2000.00, "status": "ACTIVE" } ]
```

The examples are illustrative; the console screenshot on page 11 shows an actual account creation run.

2. System Features (continued)

2.4 Deposit

POST /api/accounts/{accountNumber}/deposit
Requires a positive amount with at most two decimal places. Locks the account, checks ACTIVE status, credits the balance, and records a DEPOSIT transaction.

`{ "amount": 75.50 } → { "message": "Deposit successful." }`

2.5 Withdrawal

POST /api/accounts/{accountNumber}/withdraw
Checks account status and sufficient funds before debiting and recording a WITHDRAWAL transaction.

`{ "amount": 20.00 } → { "message": "Withdrawal successful." }`

2.6 Transfer

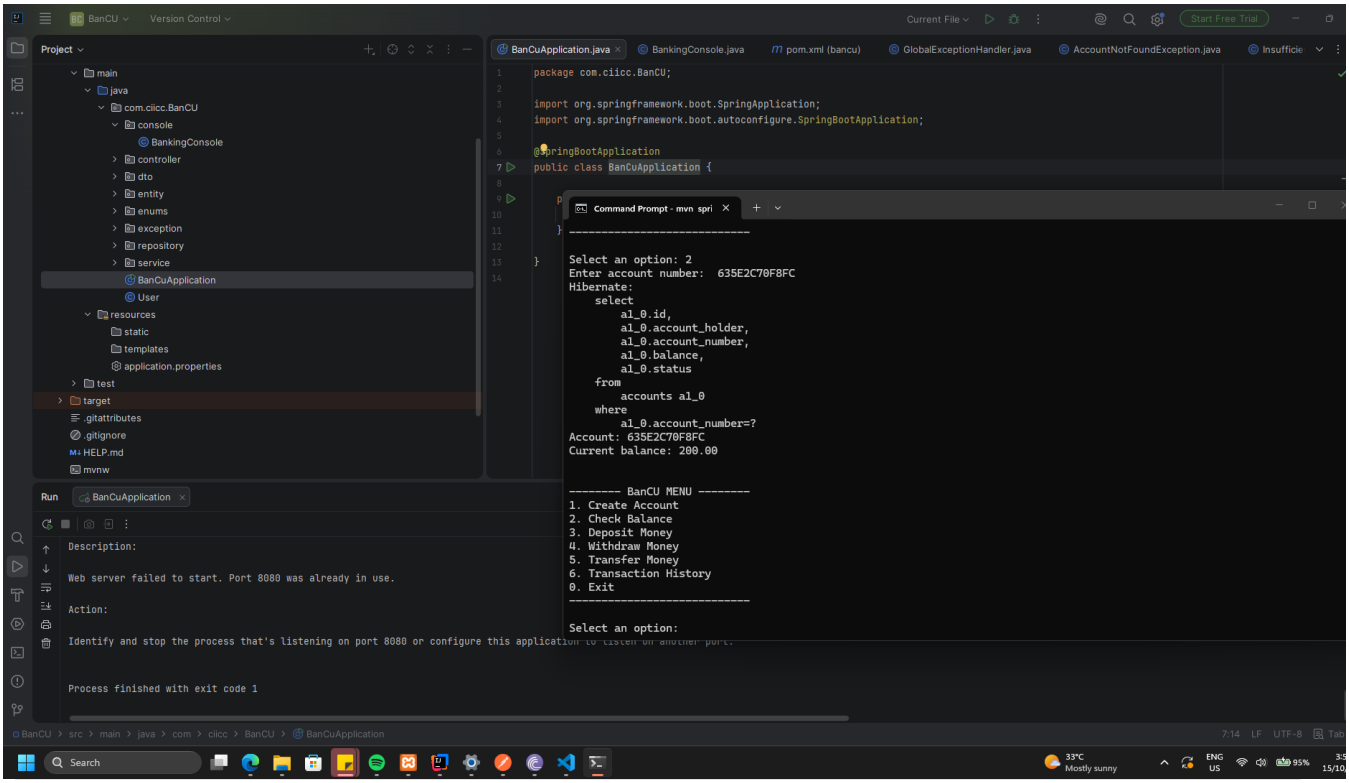
POST /api/transfers
Transfers between distinct active accounts. Locks are requested in consistent account-number order; both balances and paired transfer records are updated within one transaction.

`{ "sourceAccountNumber": "…", "destinationAccountNumber": "…", "amount": 10.00 }`

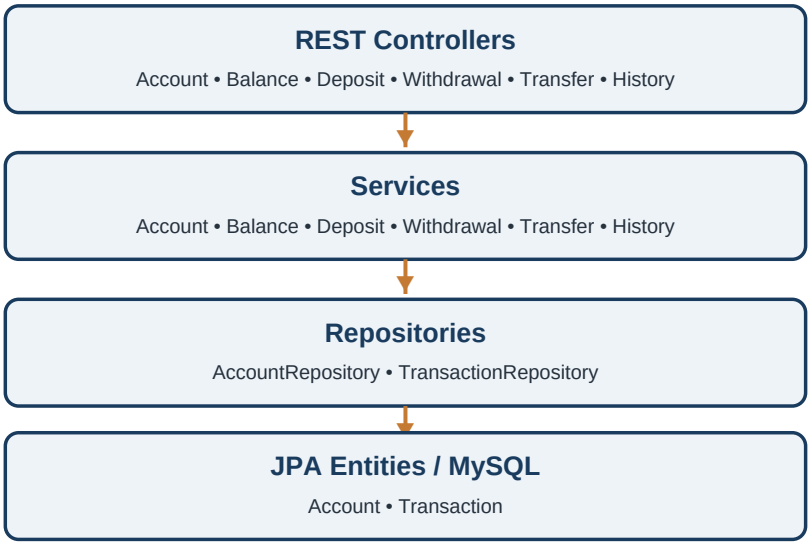
2.7 Transaction History

GET /api/accounts/{accountNumber}/transactions?page=0&size;=10
Returns newest-first paginated transaction records with reference, type, amount, balance after, related account, and timestamp. Page is zero-based; size must be 1–100.

`Response type: Page<TransactionResponse>`



3. System Design



TransactionRecorder is called by deposit, withdrawal, and transfer services to create audit records.

Major classes and responsibilities

Component	Responsibility
Controllers	Map routes, validate request DTOs, and delegate to services.
Services	Apply account, money, funds, and history rules; transactional boundaries wrap money movements.
Repositories	Provide account lookup, pessimistic locking, and paged transaction history.
Entities	Account and Transaction map to relational database records.
TransactionRecorder	Creates audit rows with post-operation balance, UUID reference, and timestamp.
GlobalExceptionHandler	Maps domain, input, and validation errors into JSON responses.
BankingConsole	Optional terminal UI, enabled by bancu.console.enabled=true.

4. Database Design

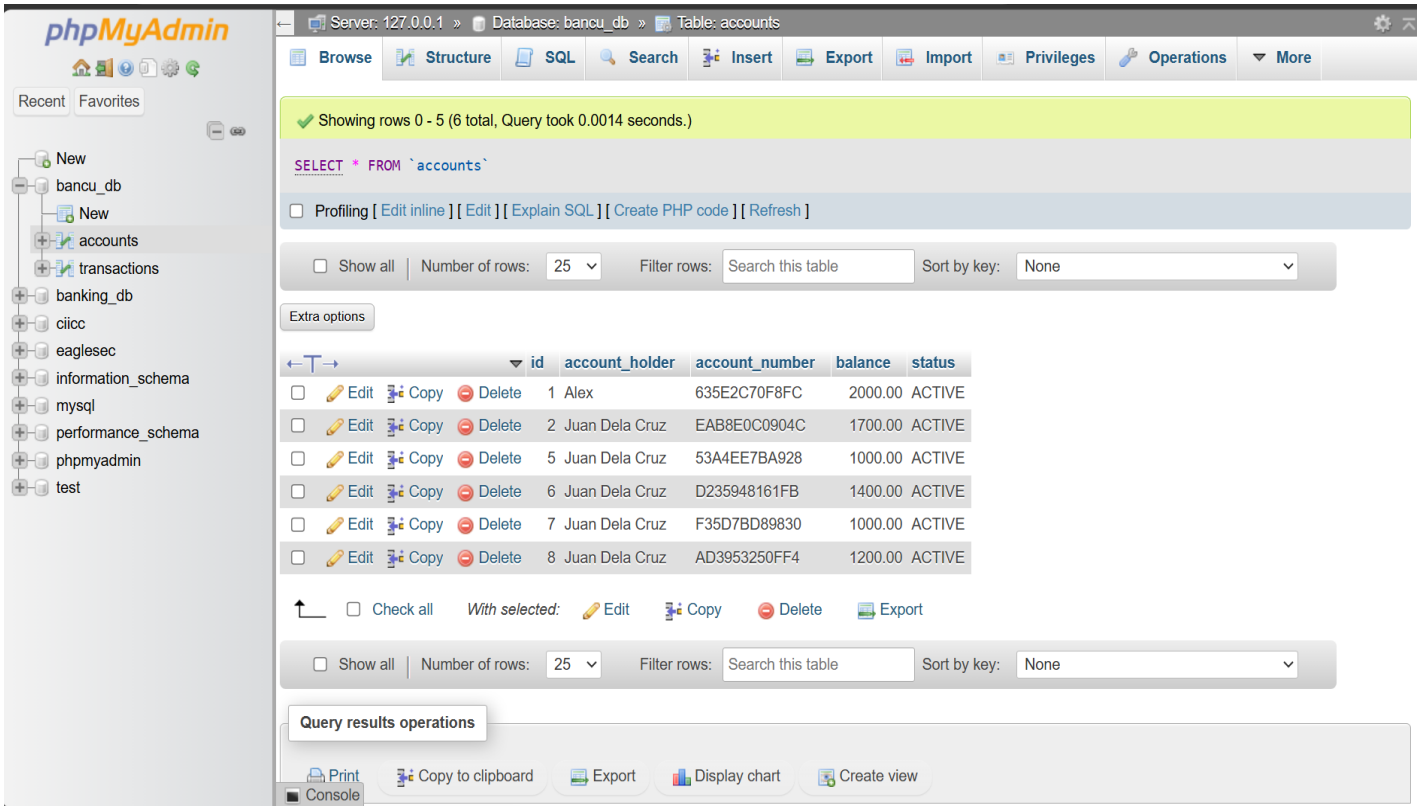
Table	Key fields / constraints
accounts	id BIGINT identity primary key; account_number VARCHAR(20), unique and not null; account_holder not null; balance DECIMAL(19,2); status enum stored as string.
transactions	id BIGINT identity primary key; account_id required FK; related_account_id nullable FK; type enum string; amount and balance_after DECIMAL(19,2); created_at; reference_number.

Relationships and configuration

Each transaction belongs to one account; an account can have many transactions. The optional related-account reference records the counterparty for transfers. Every transfer is represented by paired TRANSFER_OUT and TRANSFER_IN transaction rows.

The supplied configuration uses jdbc:mysql://localhost:3306/bancu_db, username root, an empty sample password, and server port 8080. Hibernate schema behavior is set to ddl-auto=update; SQL statement logging and formatted SQL are enabled. The console is enabled in the supplied application.properties (bancu.console.enabled=true). For deployment, move credentials into secure external configuration and review automatic schema updates and SQL logging.

Entity relationships and data precision are defined through JPA annotations. The account balance and transaction monetary columns use DECIMAL precision 19 and scale 2.



5. Code Snippets

5.1 Account listing (AccountController)

```
@GetMapping
public ResponseEntity<List<AccountResponse>> getAllAccounts() {
    return ResponseEntity.ok(accountService.getAllAccounts());
}
```

5.2 Account creation: number, opening balance, status

```
String accountNumber;
do {
    accountNumber = UUID.randomUUID().toString()
        .replace("-", "").substring(0, 12).toUpperCase();
} while (accountRepository.existsByAccountNumber(accountNumber));

BigDecimal initialDeposit = request.getInitialDeposit() == null
        ? BigDecimal.ZERO : request.getInitialDeposit();
Account account = Account.builder()
        .accountNumber(accountNumber)
        .accountHolder(request.getAccountHolder().trim())
        .balance(initialDeposit).status(AccountStatus.ACTIVE).build();
Account saved = accountRepository.save(account);
```

5.3 Deposit: locked update and audit record

```
@Transactional
public void deposit(String accountNumber, BigDecimal amount) {
    validateAmount(amount);
    Account account = accountRepository
        .findByAccountNumberForUpdate(accountNumber)
        .orElseThrow(() -> new AccountNotFoundException(accountNumber));
    if (account.getStatus() != AccountStatus.ACTIVE)
        throw new IllegalArgumentException("Account is not active.");
    account.setBalance(account.getBalance().add(amount));
    transactionRecorder.record(account, null, TransactionType.DEPOSIT, amount);
}
```

Validation note

CreateAccountRequest requires a nonblank account holder and limits initial deposit to a nonnegative value with up to two fractional digits. MoneyRequest and TransferRequest require an amount of at least 0.01, with at most two fractional digits. Service-level validation is also applied to money-changing operations.

5. Code Snippets (continued)

5.4 Withdrawal: sufficient-funds protection

```
if (account.getBalance().compareTo(amount) < 0) {
    throw new InsufficientFundsException();
}
account.setBalance(account.getBalance().subtract(amount));
transactionRecorder.record(account, null,
        TransactionType.WITHDRAWAL, amount);
```

Account is loaded with a pessimistic write lock inside a transactional service method. Insufficient funds raise a domain exception and are mapped to HTTP 400.

5.5 Transfer: paired, atomic records

```
// Accounts are locked in consistent account-number order.
source.setBalance(source.getBalance().subtract(amount));
destination.setBalance(destination.getBalance().add(amount));
transactionRecorder.record(source, destination,
        TransactionType.TRANSFER_OUT, amount);
transactionRecorder.record(destination, source,
        TransactionType.TRANSFER_IN, amount);
```

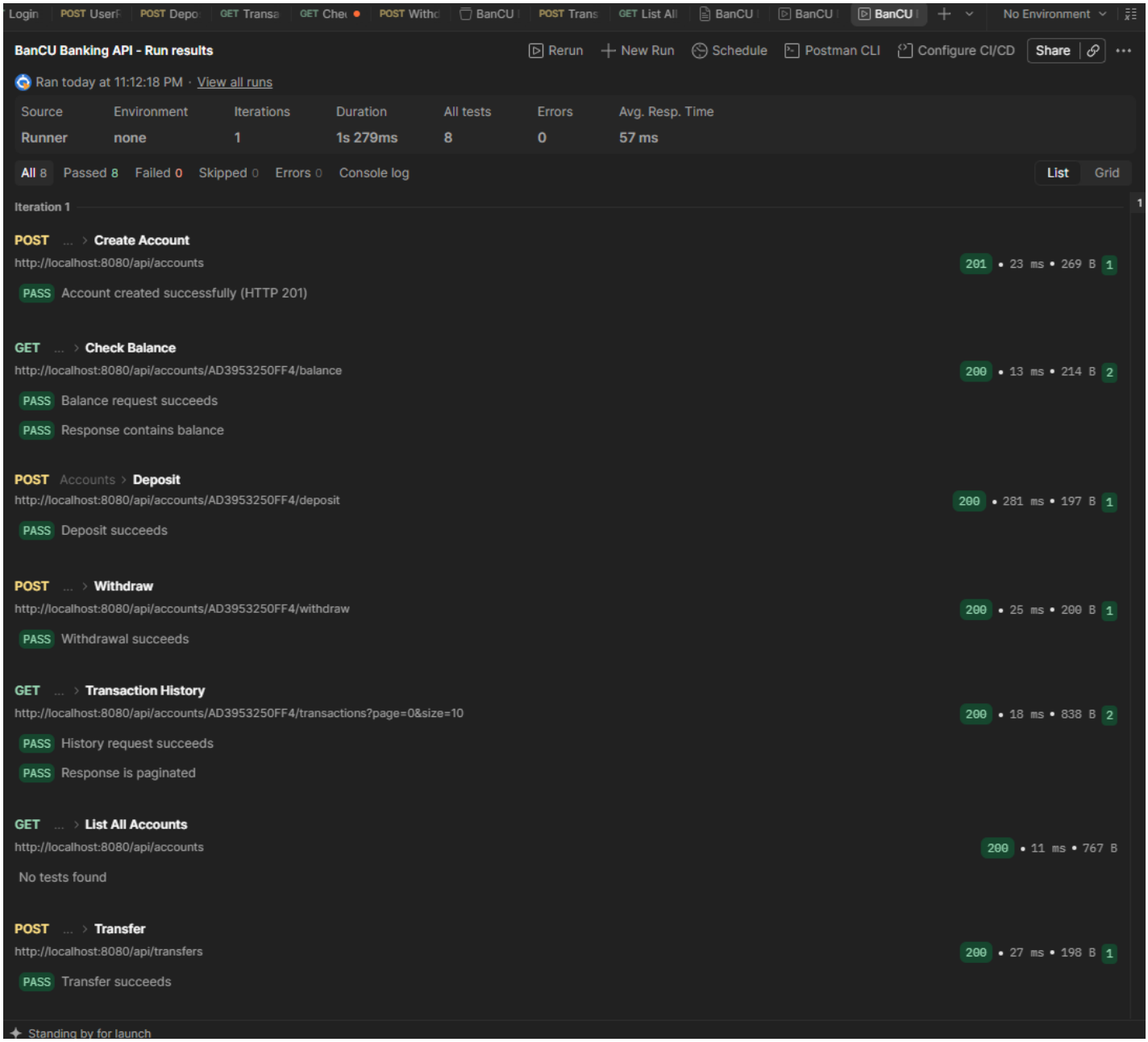
5.6 Centralized not-found response

```
@ExceptionHandler(AccountNotFoundException.class)
public ResponseEntity<Map<String, String>> handleNotFound(
        AccountNotFoundException ex) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(Map.of("error", ex.getMessage()));
}
```

GlobalExceptionHandler also maps insufficient-funds, illegal argument, and DTO validation errors to JSON error bodies. The error status is 404 for missing accounts and 400 for invalid or rejected operations.

6. Test Evidence: Postman

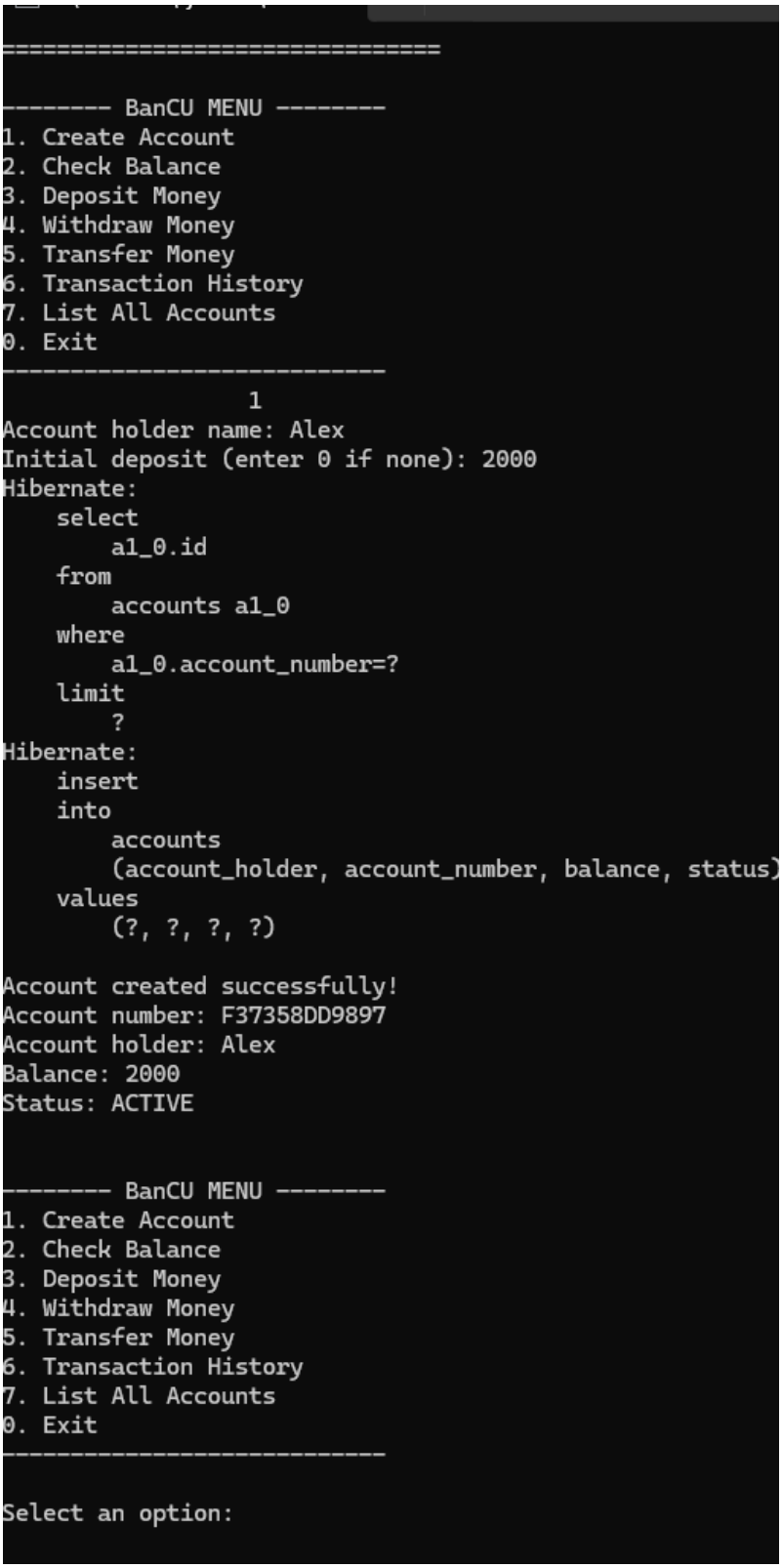
The supplied Postman collection-run screenshot displays the BanCU API run results. It records 8 tests passed, 0 failed, and 0 errors. The collection exercises account creation, balance inquiry, deposit, withdrawal, paginated history, list-all-accounts, and transfer. The list-all-accounts request returned HTTP 200; the screenshot notes that no tests were configured for that request.



Evidence image: supplied Postman run screenshot. Request results are from the user-provided test run, not generated examples.

7. Test Evidence: Banking Console

The supplied console snapshot shows the interactive Banking Console creating an account for Alex with an initial deposit of 2,000. The application reports account number F37358DD9897, balance 2000, and ACTIVE status, then returns to the menu. Hibernate SQL output is visible because SQL logging is enabled.



```
==============================

-------- BanCU MENU --------
1. Create Account
2. Check Balance
3. Deposit Money
4. Withdraw Money
5. Transfer Money
6. Transaction History
7. List All Accounts
0. Exit
--------------------------
                  1
Account holder name: Alex
Initial deposit (enter 0 if none): 2000
Hibernate:
    select
        a1_0.id
    from
        accounts a1_0
    where
        a1_0.account_number=?
    limit
        ?
Hibernate:
    insert
    into
        accounts
        (account_holder, account_number, balance, status)
    values
        (?, ?, ?, ?)

Account created successfully!
Account number: F37358DD9897
Account holder: Alex
Balance: 2000
Status: ACTIVE


-------- BanCU MENU --------
1. Create Account
2. Check Balance
3. Deposit Money
4. Withdraw Money
5. Transfer Money
6. Transaction History
7. List All Accounts
0. Exit
--------------------------

Select an option:
```

Console enabled by bancu.console.enabled=true in the supplied application.properties.

8. Conclusion & Appendix

Conclusion

BanCU implements core banking account and money-movement operations using a Spring Boot layered architecture, REST endpoints, JPA persistence, and MySQL. The code demonstrates DTO validation, BigDecimal arithmetic, transactional service operations, pessimistic locking, transaction audit records, exception handling, paginated history, and all-account listing.

Appendix A — operational notes

• Configure a reachable MySQL instance and database bancu_db before launching.
• Start the Spring Boot application on port 8080; API base URL is http://localhost:8080.
• The console is enabled by the supplied property bancu.console.enabled=true; set it to false when running as an API-only service.
• The supplied sample database credentials are development-only. Do not use blank or hard-coded credentials in deployment.
• Hibernate ddl-auto=update is used; no standalone SQL migration script was provided.

Appendix B — endpoint quick reference

Operation	Method and route
Create account	POST /api/accounts
List all accounts	GET /api/accounts
Balance inquiry	GET /api/accounts/{accountNumber}/balance
Deposit	POST /api/accounts/{accountNumber}/deposit
Withdrawal	POST /api/accounts/{accountNumber}/withdraw
Transfer	POST /api/transfers
Transaction history	GET /api/accounts/{accountNumber}/transactions?page=0&size;=10

Prepared for Alexander Inciong • Batch PRJ13 • Trainer Michael Ampo • 15 October 2026